

Celestial Mechanics Project Documentation

1. Project Overview & Basic Idea

The **Celestial Mechanics** project is a high-performance, real-time 3D simulation engine designed specifically for modeling gravitational N-body physics. The primary goal is to provide a scientifically accurate, scalable, and visually interactive desktop application to simulate cosmic phenomena ranging from simple orbital mechanics to complex planetary and stellar interactions (including collisions, post-Newtonian relativistic effects, and gravitational waves).

The system is split into two massive components:

1. A highly optimized **native C++20/CUDA backend** responsible for the heavy mathematical lifting and physics simulations.
 2. A robust **C#/ .NET 8 WPF frontend** providing a rich graphical user interface, realistic 3D OpenGL rendering, and scene orchestration.
-

2. Technological Stack

Backend (Physics Engine)

- **Language:** C++20 and CUDA.
- **Compute Models:** Multi-threaded CPU (Thread Pool / Jobs) and GPU acceleration (CUDA).
- **Core Algorithms:**
 - Octree for $O(N \log N)$ Barnes-Hut spatial partitioning.
 - Structure-of-Arrays (SoA) memory alignment for SIMD vectorization & cache-locality.
 - Post-Newtonian (1PN) Einstein-Infeld-Hoffmann corrections.

Interop Layer

- **C-API:** Exposes a pure C Application Binary Interface (ABI) relying strictly on `<stdint.h>`.
- **P/Invoke:** Bridges the C++ unmanaged memory world with the C# managed environment smoothly without overhead.

Frontend & Application Core

- **Language:** C# 12 (.NET 8).
- **UI Framework:** Windows Presentation Foundation (WPF) incorporating the `CommunityToolkit.Mvvm` pattern.
- **Rendering:** Silk.NET providing low-level OpenGL (WGL) context for hardware-accelerated 3D viewport rendering within the WPF shell.
- **Architecture:** Enforces a strict Entity-Component-System (ECS) orchestrator pattern.

Build & Testing

- **Native:** CMake.
 - **Managed:** MSBuild / `dotnet`.
 - Comprehensive Unit Testing on both C# and C++ boundaries ensuring collision detection, determinism, and mathematical precision (e.g., energy drift, moment conservation).
-

3. Architecture & Directory Structure

The repository maintains strict boundary layers avoiding circular dependencies.

Native Engine (engine/)

- **core/** : Foundation types, platform abstractions, custom exceptions, and constants (`G = 1` in N-body units).
- **math/** : Header-only deterministic 3D vectors (`vec3d`), matrices (`mat4d`), and quaternions operating in double precision.
- **job/** : Lock-free MPMC (multi-producer multi-consumer) thread pool up to 64 workers.
- **physics/** : The core logic: Barnes-Hut $O(N \log N)$ solvers, Density modeling, `ParticleSystem` (18 aligned SoA arrays including pos, vel, acc, mass, radius).
- **cuda/** : 15+ kernel files handling GPU tree-building, Morton code sorting, and prefix-scan compaction.
- **sim/ & profile/** : The `Engine` facade handling timestep accumulation, diagnostics, adaptive timestepping, and energy tracking.

Managed .NET Solution (src/)

- `CelestialMechanics.AppCore` : Handles scene graphs, project serialization (`.cesim` files), ECS entities, and event buses.
 - `CelestialMechanics.Simulation` : Thread-safe simulation looping running at ~1 kHz, completely decoupled from UI ticks.
 - `CelestialMechanics.Physics` : C# representations/configs of the physical rules mapping to the C interop.
 - `CelestialMechanics.Renderer` : The Silk.NET OpenGL abstraction, handling post-build shaders (GLSL), camera matrices, and drawing instanced spheres/trails.
 - `CelestialMechanics.Desktop` : The WPF Shell containing `App.xaml` , viewports, toolbars, and MVVM ViewModels.
-

4. Execution Plan & Simulation Pipeline

The engine leverages highly advanced integrators and parallelized dispatch sequences.

Integrators

The engine advances state using an accumulator-based loop preventing spiral-of-death (caps at 10 steps/frame).

1. **Velocity Verlet (Leapfrog)**: 2nd-order symplectic integrator using a `Kick-Drift-Kick` (KDK) topology.
2. **Yoshida 4th-Order**: Symplectic integrator composing three leapfrog substeps (where the middle step goes backward in time) to dramatically reduce global error ($O(dt^4)$).

8-Mode Engine Matrix

Simulation dispatch scales depending on user hardware availability and precision requirements:

- **CPU Brute-Force**: $O(N^2)$ pairwise gravity.
- **CPU Barnes-Hut**: $O(N \log N)$ using Octrees built on the CPU.
- **GPU Brute-Force**: Tiled shared-memory $O(N^2)$ on the GPU.
- **GPU Barnes-Hut**: $O(N \log N)$ GPU-resident tree building, traversals, and Morton sorting.

Collision Resolution

A multi-stage pipeline: **Detection** -> **Resolution** -> **Compaction**.

- Detects overlapping radii during tree traversal ($O(N \log N)$ "free" detection).
- Resolves using Elastic, Inelastic, or Merge policies.

- In GPU Merge mode, the entire detection and compaction runs resident on the GPU using `atomicAdd` and prefix-scans avoiding costly CPU round-trips.
-

5. Desktop Application Capabilities

The UI provides granular control suitable for scientific monitoring or casual sandbox creation.

UI Layout (3-Panel Dockable Grid)

1. **Scene Outliner:** TreeView to inspect nested bodies, star systems, and galaxies.
2. **3D Viewport:** Hardware-accelerated 3D view with mouse orbit, pan, and zoom mappings. Click-to-select raycasting targeting entities.
3. **Inspector & Settings:**
 - Two-way binding to tweak masses, velocities, radius, and density while paused.
 - Adjust `PhysicsConfig` (Theta thresholds for Barnes-Hut, softening epsilon, integration mode).

Key Application Features

- **Snapshot Timeline System:** Records historical step indexes. Users can scrub a slider to reverse the simulation back to prior states instantly.
 - **Event Log:** Records localized cosmic events (Supernovas, Mergers) onto a color-coded feed.
 - **Save/Load (`.cesim`):** Serialization of the entire world-state to disk via generic `ProjectSerializer`.
 - **Diagnostics:** Real-time rendering of total kinetic and potential energy, ensuring energy drift remains within microscopic constraints (vital for N-body accuracy validation).
-

Generated mathematically and algorithmically accurate representation of the Celestial Mechanics ecosystem.